`acceleratum` package overview

Contents

1 Scope of the package **1**

2 Data format and handling **2**

2.1 The `aclrtm_accelerometery` class 2

2.2 The `aclrtm_burst` class 4

3 Calibration **6**

3.1 Known position filtering with `segment_select()` 7

3.2 Parameter estimation 10

3.3 Apply calibration 12

3.4 Unknown position filtering 14

4 Standardise rotation **16**

4.1 Rotate 2 axes: the collar case 16

4.2 Rotation in 3 dimensions 18

5 References **19**

1 Scope of the package

`acceleratum` has been developed to aid with bench calibration of (triaxial) accelerometer devices prior to field deployment. While originally developed and used in a biologging context, its application are not restricted to it.

A triaxial accelerometer needs calibration to correct systematic errors that prevent raw measurements from matching true acceleration. In `acceleratum` we consider three sources of errors that are structural to the device:

- Scale (S): Each axis may respond differently to the same acceleration (e.g., +1g on the x-axis might read as 0.98, while +1g on y reads as 1.02). Calibration equalizes these sensitivities.
- Misalignment (M): The sensor's internal axes may not be perfectly orthogonal (90° apart). This causes acceleration along one axis to leak into another.
- Bias (b): Each axis can have a constant offset (e.g., reading 0.1g even at rest), which must be subtracted to center measurements at zero.

`acceleratum` also allows to correct for mispositioning of a deployed device due to user error (e.g. a device mounted backwards), and unavoidable small differences in positioning of a device deployed on living animals.

While bench calibration of a device is preferred, `acceleratum` also provides methods to attempt the estimation of calibration parameters from deployed devices.

```
library("acceleratum")
```

2 Data format and handling

Functions dealing with parameter estimation are time-agnostic. As such, these were written to handle data in matrix format.

2.1 The `aclrtm_accelerometery` class

To make it easier to visualise and print matrices of accelerometer data, `acceleratum` provides a helper class which use is largely optional: `aclrtm_accelerometery`. An `aclrtm_accelerometery` can be initialised using the function `accelerometery()`

```
set.seed(42)
some_data <- rnorm(90)
head(some_data)
#> [1]  1.3709584 -0.5646982  0.3631284  0.6328626  0.4042683 -0.1061245
```

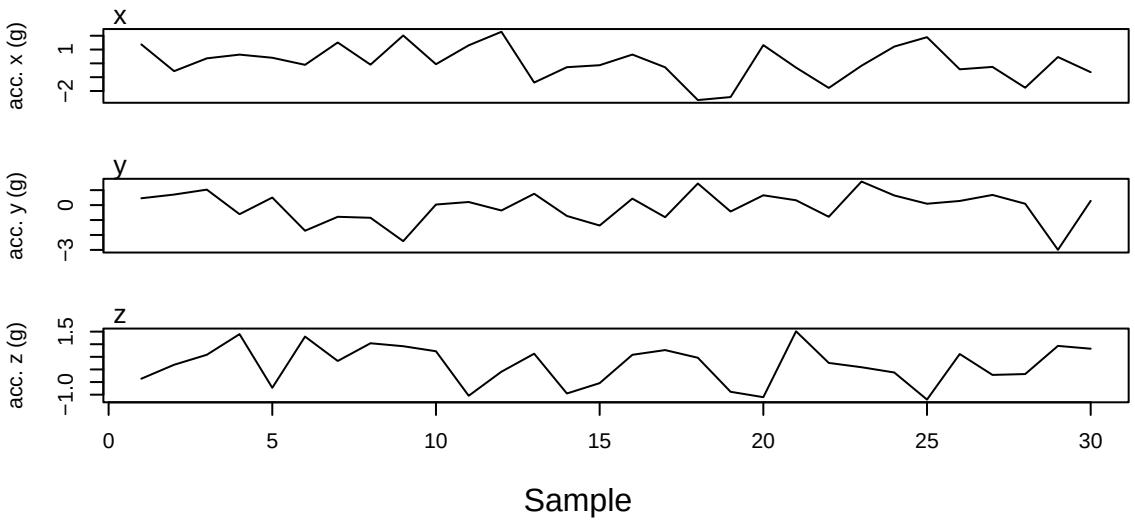
```
# aclrtm_accelerometery from a numeric vector assumes repeating x, y, z order:
# x1, y1, z1, x2, y2, z2, ... , xn, yn, zn
acc <- accelerometery(some_data)
#> Note: `axes` not supplied; inferred axes = "xyz".
head(acc)
#> <aclrtm_accelerometery>  [6 samples × 3 axes (xyz)]
#>               x           y          z
#> [1,]  1.3709584 -0.56469817  0.3631284
#> [2,]  0.6328626  0.40426832 -0.1061245
#> [3,]  1.5115220 -0.09465904  2.0184237
#> [4,] -0.0627141  1.30486965  2.2866454
#> [5,] -1.3888607 -0.27878877 -0.1333213
#> [6,]  0.6359504 -0.28425292 -2.6564554
```

```
# aclrtm_accelerometery from matrix
# Note how the vector was now arranged column-wise. That's due to the behaviour
# of matrix() that defaults to byrow = FALSE
some_data <- matrix(some_data, ncol = 3)
acc <- accelerometery(some_data)
#> Note: `axes` not supplied; inferred axes = "xyz".
head(acc)
#> <aclrtm_accelerometery>  [6 samples × 3 axes (xyz)]
#>               x          y          z
#> [1,]  1.3709584  0.4554501 -0.3672346
#> [2,] -0.5646982  0.7048373  0.1852306
#> [3,]  0.3631284  1.0351035  0.5818237
#> [4,]  0.6328626 -0.6089264  1.3997368
```

```
#> [5,]  0.4042683  0.5049551 -0.7272921
#> [6,] -0.1061245 -1.7170087  1.3025426
```

```
# aclrtm_accelerometery from data.frame
some_data_df <- as.data.frame(some_data)
colnames(some_data_df) <- c("x", "y", "z")
acc <- accelerometery(some_data_df)
head(acc)
#> <aclrtm_accelerometery>  [6 samples × 3 axes (xyz)]
#>               x          y          z
#> [1,]  1.3709584  0.4554501 -0.3672346
#> [2,] -0.5646982  0.7048373  0.1852306
#> [3,]  0.3631284  1.0351035  0.5818237
#> [4,]  0.6328626 -0.6089264  1.3997368
#> [5,]  0.4042683  0.5049551 -0.7272921
#> [6,] -0.1061245 -1.7170087  1.3025426
```

```
# Plotting method for aclrtm_accelerometery objects
plot(acc)
```



Optionally, arguments `sampling_rate` and `start_time` can be provided and will be set as attributes. When available, subsetting rows will account for the shift.

```
acc <- accelerometery(some_data,
                      axes = "xyz",
                      sampling_rate = 4, # in Hz
                      start_time = as.POSIXct("2026-01-01 10:00:00",
                                              tz = "UTC"))
head(acc, 3)
#> <aclrtm_accelerometery>  [3 samples × 3 axes (xyz)]
#>   sampling_rate : 4 Hz
#>   start_time : 2026-01-01 10:00:00
#>               x         y          z
#> [1,]  1.3709584 0.4554501 -0.3672346
```

```
#> [2,] -0.5646982 0.7048373  0.1852306
#> [3,]  0.3631284 1.0351035  0.5818237
```

```
acc[20:23,]
#> <aclrtm_accelerometery>  [4 samples × 3 axes (xyz)]
#>   sampling_rate : 4 Hz
#>   start_time : 2026-01-01 10:00:04
#>                x          y           z
#> [1,]  1.3201133  0.6556479 -1.09978090
#> [2,] -0.3066386  0.3219253  1.51270701
#> [3,] -1.7813084 -0.7838389  0.25792144
#> [4,] -0.1719174  1.5757275  0.08844023
```

```
acc[c(10, 14, 17),]
#> Note: row subsetting is not sequential. start_time and sampling_rate have been dropped.
#> <aclrtm_accelerometery>  [3 samples × 3 axes (xyz)]
#>                x           y          z
#> [1,] -0.0627141  0.03612261  0.7208782
#> [2,] -0.2787888 -0.72670483 -0.9535234
#> [3,] -0.2842529 -0.81139318  0.7681787
```

When constructing from a data frame, a timestamp column can be provided

```
some_data_df$timestamp <- seq(as.POSIXct("2026-01-01 10:00:00",
                                         tz = "UTC"),
                              by = 1/4,
                              length.out = nrow(some_data_df))

acc <- accelerometery(some_data_df,
                      ts_col = "timestamp")
#> Note: `sampling_rate` estimated as 4 Hz.
head(acc)
#> <aclrtm_accelerometery>  [6 samples × 3 axes (xyz)]
#>   sampling_rate : 4 Hz
#>   start_time : 2026-01-01 10:00:00
#>                x          y          z
#> [1,]  1.3709584  0.4554501 -0.3672346
#> [2,] -0.5646982  0.7048373  0.1852306
#> [3,]  0.3631284  1.0351035  0.5818237
#> [4,]  0.6328626 -0.6089264  1.3997368
#> [5,]  0.4042683  0.5049551 -0.7272921
#> [6,] -0.1061245 -1.7170087  1.3025426
```

Note that an `aclrtm_accelerometery` object uses attributes `sampling_rate` and `start_time` just as an aid in plotting, but does not store the timestamp of each observation and assumes consistent sampling rate across the data. If your data comes from deployments that cannot safely assume consistent sampling rate, some precautions must be taken to align the output of calibration functions with the correct timestamp stored elsewhere. More on that later.

2.2 The `aclrtm_burst` class

Sometimes, accelerometer data is stored on file in "bursts", with multiple observations per row. `acceleratum` provides the helper class `aclrtm_burst` to help with management of such data. Initialised via `burst()`:

```
data(bench) # some data in burst format
```

An extract from `help(bench)`:

> The `accelerations_raw` column stores the data in bursts as a single space-separated character string, with axis measurements interleaved in repeating x, y, z order:
>
> `"x1 y1 z1 x2 y2 z2 ... xn yn zn"`
>
> where `n` is the number of samples in that burst.

```
brst <- burst(bench, "accelerations_raw", ts_col = "timestamp")
#> Note: `axes` not supplied; inferred axes = "xyz".
```

```
head(brst)
#> <aclrtm_burst>  [6 rows x 6 cols]
#>   data_col : "accelerations_raw"
#>   axes     : "xyz"
#>   ts_col   : "timestamp"
#>   tag_local_identifier individual_local_identifier           timestamp
#> 1                26147                         cal 2025-02-21 14:55:26
#> 2                26147                         cal 2025-02-21 14:55:31
#> 3                26147                         cal 2025-02-21 14:55:36
#> 4                26147                         cal 2025-02-21 14:55:41
#> 5                26147                         cal 2025-02-21 14:55:46
#> 6                26147                         cal 2025-02-21 14:55:51
#>   acceleration_axes acceleration_sampling_frequency_per_axis accelerations_raw
#> 1               XYZ                                       8     <matrix 40x3>
#> 2               XYZ                                       8     <matrix 40x3>
#> 3               XYZ                                       8     <matrix 40x3>
#> 4               XYZ                                       8     <matrix 40x3>
#> 5               XYZ                                       8     <matrix 40x3>
#> 6               XYZ                                       8     <matrix 40x3>
```

A print method is available for `aclrtm_burst` objects and a `write_burst()` function is provided to write burst objects to external files.

An `aclrtm_burst` object can then be converted to `aclrtm_accelerometery` using `burst_to_accelerometery()` or its alias `b2a()`

```
acc <- b2a(brst)
head(acc)
#> <aclrtm_accelerometery>  [6 samples × 3 axes (xyz)]
#>   sampling_rate : 8 Hz
#>   start_time : 2025-02-21 14:55:26
#>      x    y     z
#> [1,] 0 0.06 -0.84
#> [2,] 0 0.06 -0.84
#> [3,] 0 0.06 -0.84
#> [4,] 0 0.06 -0.84
#> [5,] 0 0.06 -0.84
#> [6,] 0 0.06 -0.84
```

and the inverse operation is possible with `accelerometery_to_burst()` or its alias `a2b()`

```
brst_new <- a2b(acc, 5)
head(brst_new)
#> <aclrtm_burst>  [6 rows x 2 cols]
#>   data_col : "burst"
#>   axes     : "xyz"
#>   ts_col   : "timestamp"
#>              timestamp         burst
#> 1 2025-02-21 14:55:26 <matrix 40x3>
#> 2 2025-02-21 14:55:31 <matrix 40x3>
#> 3 2025-02-21 14:55:36 <matrix 40x3>
#> 4 2025-02-21 14:55:41 <matrix 40x3>
#> 5 2025-02-21 14:55:46 <matrix 40x3>
#> 6 2025-02-21 14:55:51 <matrix 40x3>
```

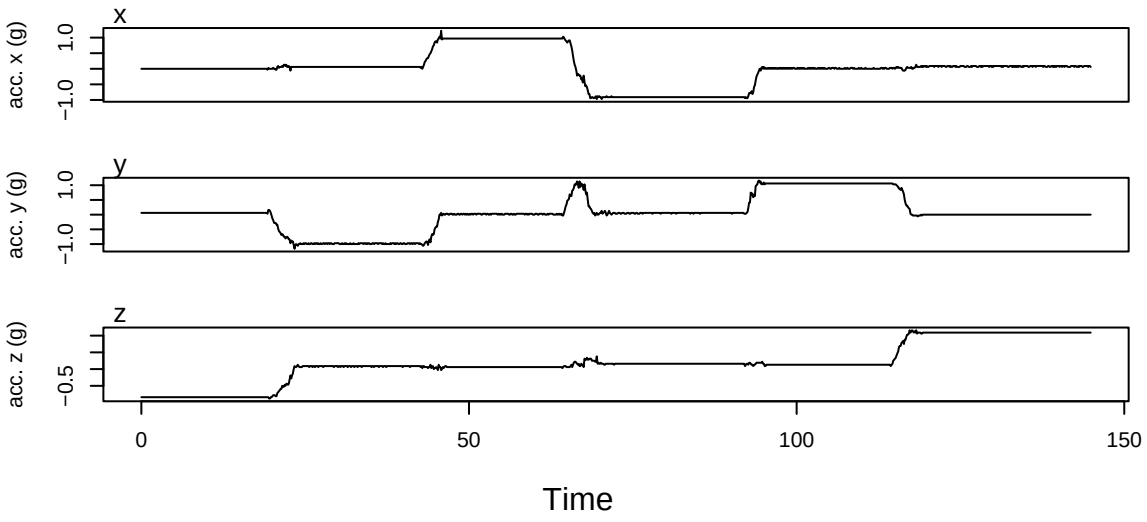
Note that converting an `aclrtm_burst` object to `aclrtm_accelerometery` and back again is not a lossless round-trip. As noted above, an `aclrtm_accelerometery` object does not store individual timestamps. Consequently, if the original aclrtm_burst object had irregular timestamps (e.g. variable gaps between bursts) or variable burst sizes (i.e. ragged bursts), that information is not recoverable from the `aclrtm_accelerometery` object alone. The reconstructed burst object will have uniformly spaced timestamps and equal-sized bursts (except possibly the last), regardless of the original structure. All additional columns present in the original object will also be lost.

3 Calibration

To showcase how to perform calibration parameter estimation from bench acquired data we will use the `bench` dataset. The `bench` dataset includes data from a triaxial accelerometer device mounted on a cubic frame and rotated through 6 known orientations over about 3 minutes. Sampling rate is fixed at 8 Hz. The raw data is stored in burst format (see the previous chapter) in column `accelerations_raw`. You can consult more information with `help(bench)`.

```
data(bench, package = "acceleratum")
acc <- b2a(burst(bench, "accelerations_raw", "xyz", "timestamp"))
```

```
plot(acc)
```

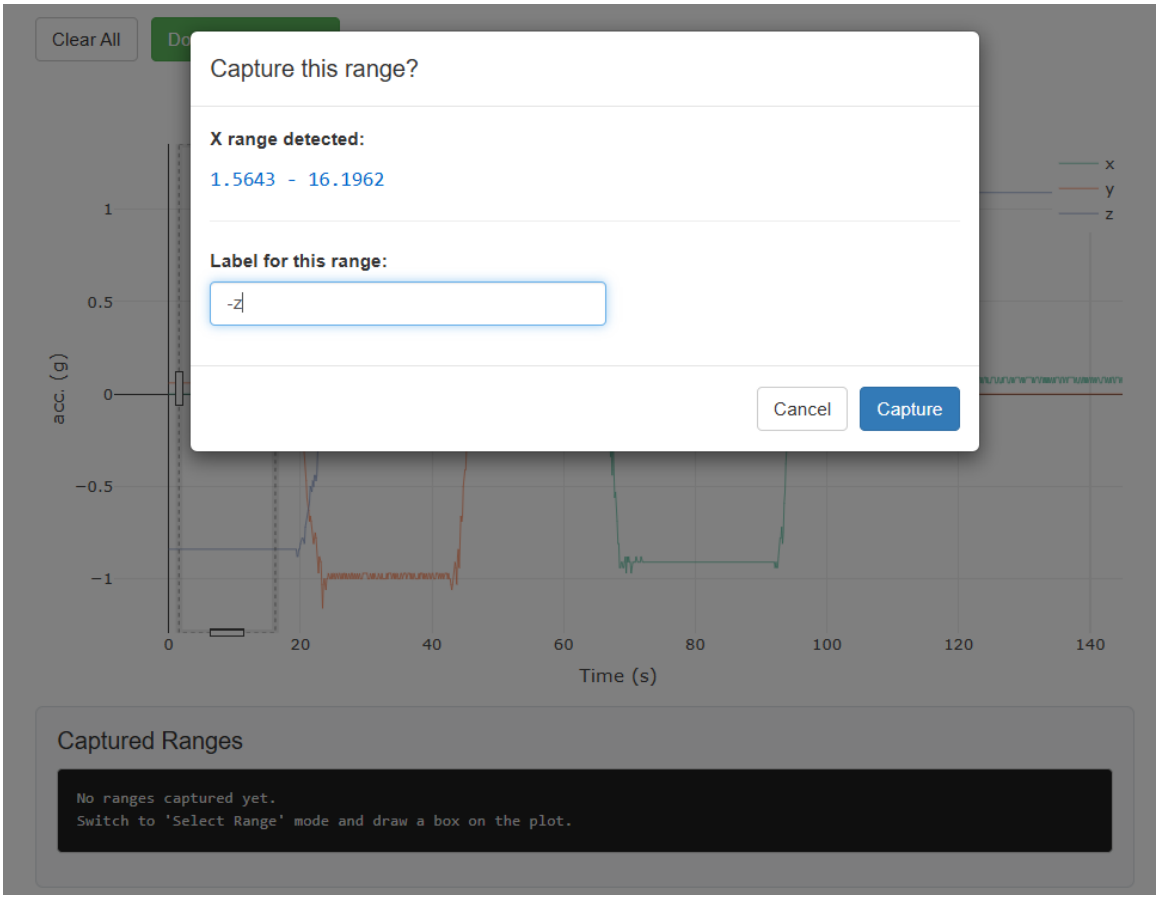


3.1 Known position filtering with `segment_select()`

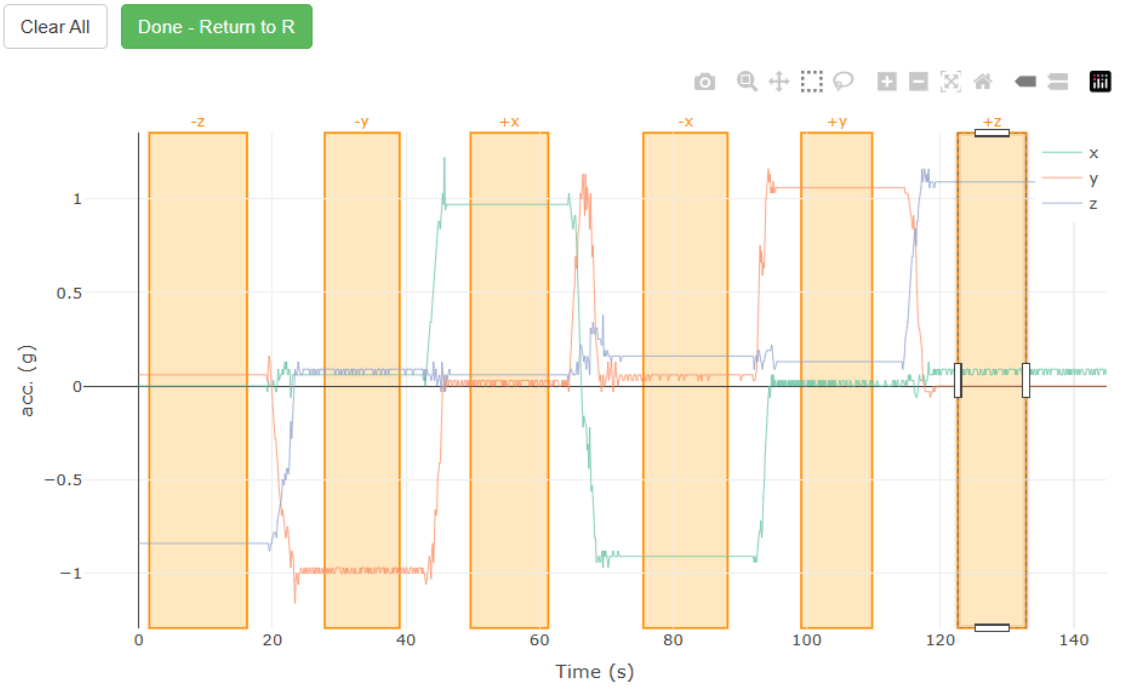
The easiest way to perform the calibration is to manually select segments in the data for which the direction of the gravity vector is known. That can be done interactively with `segment_select()`, which will open a shiny app locally

```
bench_annotations <- segment_select(acc)
```

The user can click and drag to select (horizontal) segments of interest and label them



Then repeat for all other segments.



Click "Done" to close the app and output the result. For ease of use, we included a small dataset of annotations taken with `segment_select()` on the `bench` dataset:

```
data(bench_annotations, package = "acceleratum")
bench_annotations
#>   label      xmin      xmax imin imax
#> 1    -z   1.42242  14.53693   13  117
#> 2    -y  24.62502  40.16067  199  322
#> 3    +x  49.84523  61.74917  400  494
#> 4    -x  72.84607  89.59229  584  717
#> 5    +y  98.46981 112.39136  789  900
#> 6    +z 123.08473 137.20805  986 1098
```

Annotations should then be processed on the data to be used as input into the calibration function.

```
seg_list <- process_annotations(acc, bench_annotations)
```

3.2 Parameter estimation

`cal_accel()` is the main function to estimate calibration parameters given a list of segments (such as the output of `process_annotations()`).

```
cal_param <- cal_accel(seg_list)
#>   [LS] iter    1: dP = 1.1460e-01
#>   [LS] iter    2: dP = 0.0000e+00
#>   [LS] converged in 2 iterations
```

```
cal_param
#> $S
#>           [,1]     [,2]     [,3]
#> [1,] 0.9412081 0.000000 0.000000
#> [2,] 0.0000000 1.022961 0.000000
#> [3,] 0.0000000 0.000000 0.966527
#>
#> $M
#>             [,1]        [,2]        [,3]
#> [1,]  0.99871643 -0.02691576  0.04290721
#> [2,] -0.01828330  0.99940266 -0.02932663
#> [3,] -0.05173161  0.02193420  0.99842012
#>
#> $bias
#> [1] 0.03501709 0.03566748 0.11460000
#>
#> $iters
#> [1] 2
#>
#> $deltaP
#> [1] 0
#>
#> $beta_deg
#> [1] NA
```

Optionally, if all 6 "pure" orientations for a triaxial accelerometer have been taken (and labelled correctly in the segment list), `cal_accel()` allows for the simultaneous estimation of calibration parameter and the tilt of the bench platform following the closed-form calibration method by Xu et al. (2021).

```
cal_param <- cal_accel(seg_list, estimate_tilt = TRUE)
#>   [Xu] iter    1: deltaP = 1.0224e+00
#>   [Xu] iter    2: deltaP = 1.5381e-03
#>   [Xu] iter    3: deltaP = 4.6072e-06
#>   [Xu] iter    4: deltaP = 3.7873e-08
#>   [Xu] iter    5: deltaP = 7.9748e-10
#>   [Xu] converged in 5 iterations
```

```
cal_param
#> $S
#>           [,1]     [,2]      [,3]
#> [1,] 0.9412144 0.000000 0.0000000
#> [2,] 0.0000000 1.022968 0.0000000
#> [3,] 0.0000000 0.000000 0.9665335
#>
#> $M
#>             [,1]        [,2]        [,3]
#> [1,]  0.99871643 -0.02691576  0.04290721
#> [2,] -0.01828330  0.99940266 -0.02932663
```

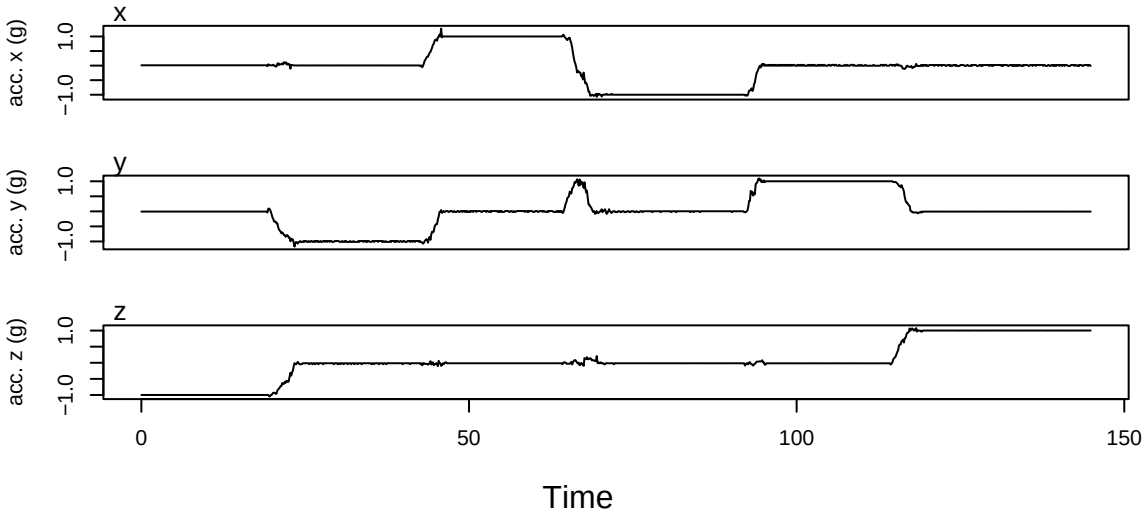
```
#> [3,] -0.05173161  0.02193420  0.99842012
#>
#> $bias
#> [1] 0.02990698 0.03771867 0.12507784
#>
#> $iters
#> [1] 5
#>
#> $deltaP
#> [1] 7.974846e-10
#>
#> $beta_deg
#> [1] -0.210377
```

3.3 Apply calibration

The raw data can then be calibrated by:

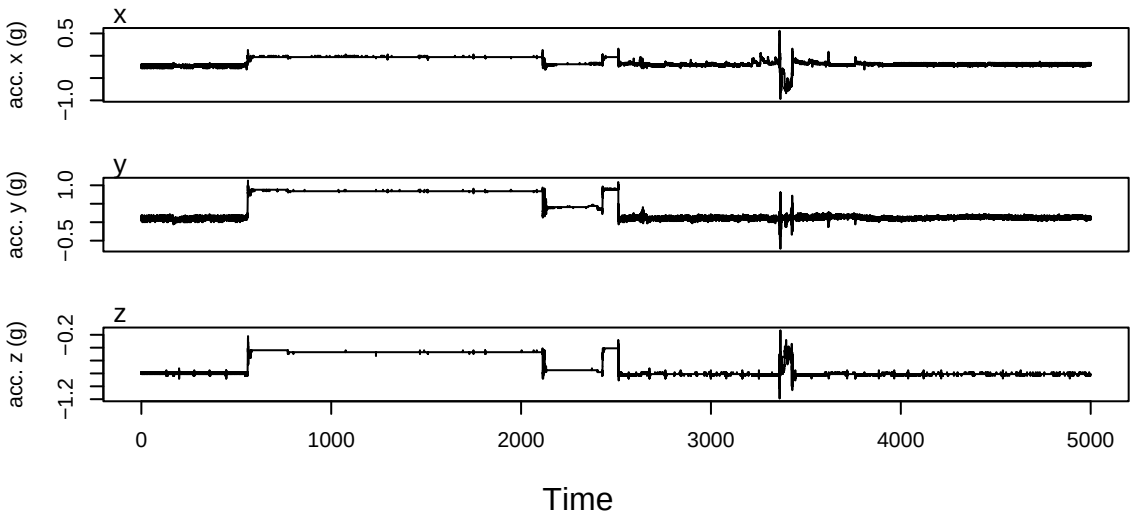
```
acc_calib <- apply_cal(acc,
                       S = cal_param$S,
                       M = cal_param$M,
                       b = cal_param$bias)

plot(acc_calib) # compare with previous
```

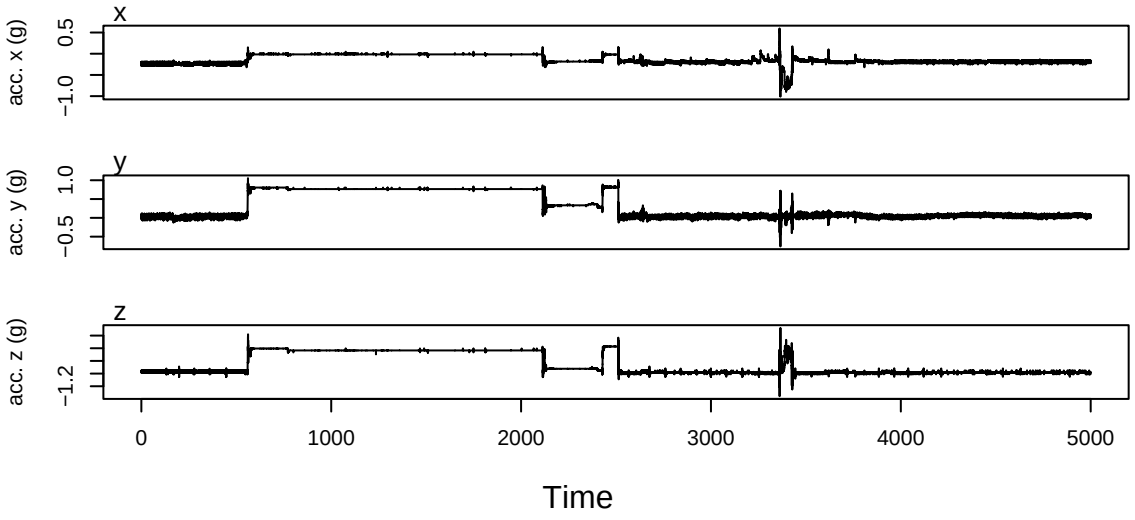



```
# example on field data collected with the same device
data(muskox)

brst_ox <- burst(muskox[1:1000,], # subset rows for showing purposes
                 "accelerations_raw", "xyz", "timestamp")
plot(brst_ox)
```

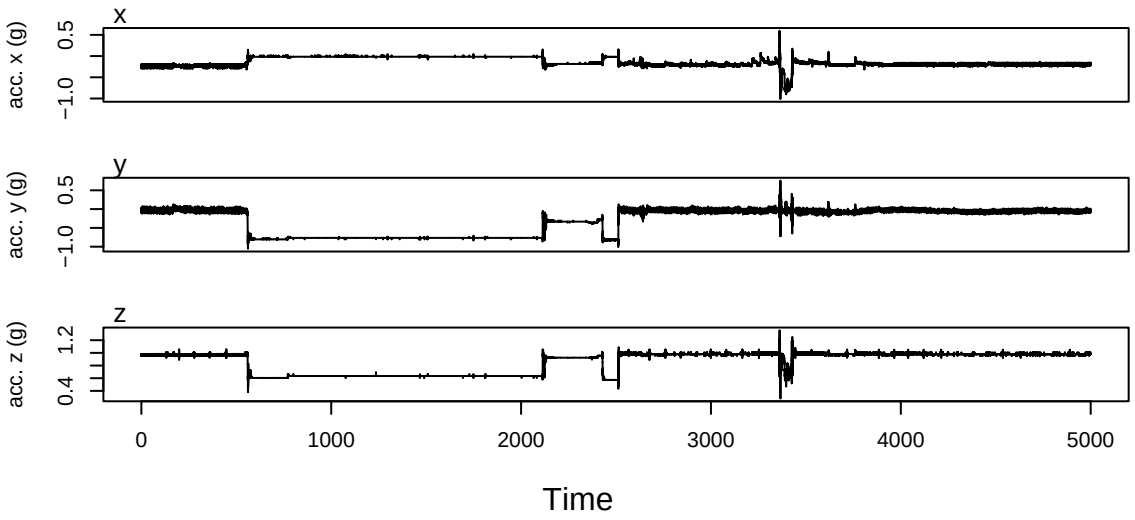



```
# apply_cal works on aclrtm_burst objects
brst_ox_calib <- apply_cal(brst_ox,
                           S = cal_param$S,
                           M = cal_param$M,
                           b = cal_param$bias)
plot(brst_ox_calib)
```



Additionally, with `apply_cal()` it is possible to flip around one axis to account for a misplaced device. Say, for example, that the device was placed upside down (rotated 180 degrees around axis x) at deployment:

```
brst_ox_calib_flipped <- apply_cal(brst_ox,
                                   S = cal_param$S,
                                   M = cal_param$M,
                                   b = cal_param$bias,
                                   R = "x")
plot(brst_ox_calib_flipped)
```



3.4 Unknown position filtering

`acceleratum` has a routine to attempt to calibrate devices that were deployed and for which bench calibration is not possible.

Note: be aware that these functions are still experimental and are quite slow.

This routine currently works exclusively with data in `burst` format and allows for the processing of large data files in chunks, as long as these are saved in burst-like format (e.g. using `write_burst()`).

The pipeline includes three steps in three functions:

- `mine_reservoir()`: reservoir sampling of windows of data which show little movement activity (as controlled by arguments `vedba_thresh`, `var_thresh`, `mag_thresh`). The function stores the data of each sampled window in a scratch binary file. The mean vectors of the windows are returned.
- `select_fps()`: performs furthest point sampling of `k` windows using their means (output of `mine_reservoir()`) and return their indexes.
- `reconstruct_selected()`: from the reservoir information stored in the output of `mine_reservoir()` and the indexes of the selected window, recreates a list of data windows to be input into `cal_accel()`.

For simplicity, a wrapper for the three steps is provided: `mine_and_select()`.

```
# can run the same example with data(muskox), but it would be noticeably
# slower. For the purpose of the example, we are sticking to the
# bench data.

data(bench)
bench <- burst(bench,
               data_col = "accelerations_raw",
               # a current limitation to our functions is that
               # a timestamp column must be defined
               ts_col = "timestamp")
#> Note: `axes` not supplied; inferred axes = "xyz".
```

```
reservoir <- mine_reservoir(
  bench,
  reservoir_size = 100
)
#> Note: few samples in the first chunk, the estimated sampling rate (8 Hz) might be unreliable. Provid
```

```
selected_points <- select_fps(reservoir, k = 6)
```

```
seg_list_unknown <- reconstruct_selected(reservoir,
                                         selected_points$selected_idx)
```

`mine_and_select()` is a useful wrapper for the last three steps

```
seg_list_unknown_alt <- mine_and_select(
  bench,
  k = 6,
  reservoir_size = 100
)
#> Note: few samples in the first chunk, the estimated sampling rate (8 Hz) might be unreliable. Provid

all.equal(seg_list_unknown_alt, seg_list_unknown)
#> [1] TRUE
```

Note that, due to further point sampling approach and the unknown direction of the real gravity vector, the misalignment matrix cannot be reliably estimated.

```
cal_param_unknown <- cal_accel(seg_list_unknown,
                               diagonal_only = TRUE,
                               verbose = FALSE)
#>   [LS] converged in 39 iterations
```

```
cal_param_unknown
#> $S
#>           [,1]    [,2]      [,3]
#> [1,] 0.9433703 0.00000 0.0000000
#> [2,] 0.0000000 1.02432 0.0000000
#> [3,] 0.0000000 0.00000 0.9663367
#>
#> $M
#>      [,1] [,2] [,3]
#> [1,]    1    0    0
#> [2,]    0    1    0
#> [3,]    0    0    1
#>
#> $bias
#> [1] 0.03118990 0.03603895 0.12554379
#>
#> $iters
#> [1] 39
#>
#> $deltaP
#> [1] 9.431403e-09
```

```
#>
#> $beta_deg
#> [1] NA
```

```
bench_cal_unknown <- apply_cal(bench,
                               S = cal_param_unknown$S,
                               M = cal_param_unknown$M,
                               b = cal_param_unknown$bias)
```

4 Standardise rotation

In `acceleratum` we provide functions to standardise the rotation of devices across multiple deployments. The approach rests on the assumption that subjects across deployments behave similarly enough that the direction of the peak density vector consistently represents the same direction relative to the subjects' reference frame. For most cases, that should correspond to the direction of gravity during the subjects' most common posture.

Under this assumption, observed differences in the direction of the peak density vector are a consequence of differences in positioning of the device. Aligning each deployment's peak density vector to a common reference vector removes this source of variation.

However, aligning a single axis still allows for infinite rotational configurations. To fully fix the orientation, a secondary axis can optionally be specified. The secondary alignment targets the direction of maximum (or minimum) density in the plane perpendicular to the primary axis, constraining the remaining rotational degree of freedom.

Constraining the rotation to the perpendicular plane is also useful when one axis can be assumed fixed (e.g. a device rigidly mounted on a collar), in which case the rotational degrees of freedom reduce to a single angle. Alignment can then be performed exclusively by rotating the plane perpendicular to the fixed axis, making the secondary alignment step unnecessary.

Note: all rotation-related operations should be carried after calibration.

4.1 Rotate 2 axes: the collar case

The following example will be using the `muskox` data. The `muskox` data was collected with a triaxial accelerometer placed on a collar. The collar used also carries a battery that acts as counterweight and is, when the animal is in a neutral standing position, placed in the lower part of the collar, towards the ground. The accelerometer is instead placed approximatively opposite of the battery, but due to interindividual differences in how tight the collar is deployed, the position of the accelerometer can rotate slightly compared to an ideal placement.

Because the accelerometer is built on the collar with the x axis facing the same direction of the animal, we assume variations along the x axis due to slight changes in the tightness of the collar to be negligible. Instead, we want to correct for the possible changes in the y- and z- axes.

```
data(muskox, package = "acceleratum")

acc <- muskox |>
  burst("accelerations_raw", "xyz", "timestamp") |>
  b2a()

# downsampling to 0.5 Hz
acc_ds <- downsample(acc, to_sr = 0.5)
```

In the ideal placement, the z-axis is aligned with the gravity vector, but with inverted directionality (i.e. negative values should point towards the ground). As such, we expect the vector of peak density to be approximatively

$$\overrightarrow{v} = (0, -1)$$

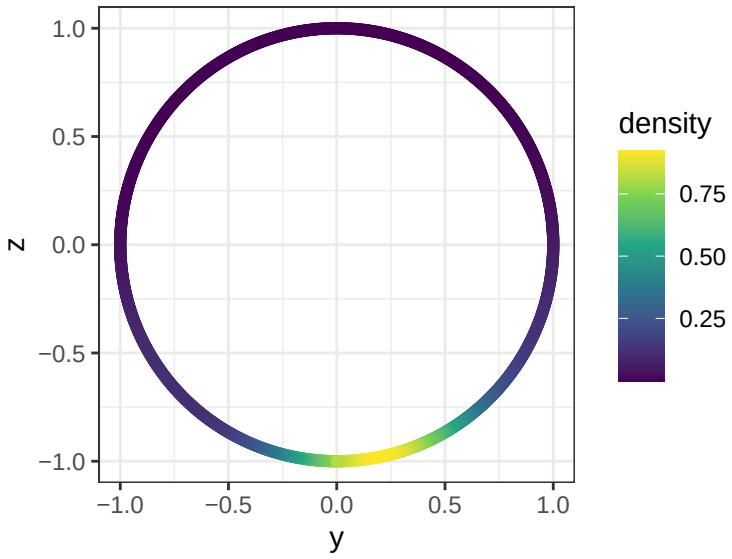
in the yz-plane.

We can explore the directional density using `vmf_kde()`

```
kde <- vmf_kde(acc_ds[,c("y", "z")])
```

```
# load ggplot2 package for visualisation
library("ggplot2")
```

```
as.data.frame(kde) |>
  ggplot(aes(y, z, colour = density)) +
  geom_point() +
  scale_colour_viridis_c() +
  coord_fixed() +
  theme_bw()
```



We notice that the density peak is off-centered compared to the expected. We then use the `rotation_to_align()` function to find the rotation to apply to the input data to have it aligned with the expected values.

```
R <- rotation_to_align(acc_ds,
                       align_to = "-z",
                       fixed_ax = "x",
                       n_grid = 360*3)

R
#>   x          y         z
#> x 1  0.0000000 0.0000000
```

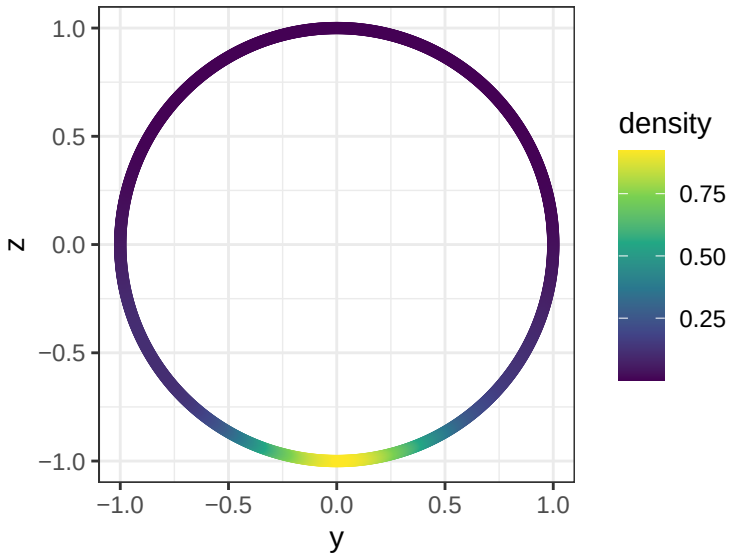
```
#> y 0  0.9837808 0.1793746
#> z 0 -0.1793746 0.9837808
```

The rotation can then be applied:

```
acc_ds_rot <- acc_ds |>
  apply_rotation(R)
```

And we can compare the result by plotting:

```
acc_ds_rot[,c("y", "z")] |>
  vmf_kde() |>
  as.data.frame() |>
  ggplot(aes(y, z, colour = density)) +
  geom_point() +
  scale_colour_viridis_c() +
  coord_fixed() +
  theme_bw()
```



4.2 Rotation in 3 dimensions

The steps are similar for a rotation in 3 dimension. Without specifying a secondary axis to fix the rotational configuration, a single step rotation, consisting of a Rodrigues rotation, is used.

```
R <- rotation_to_align(acc_ds,
                       align_to = "-z",
                       n_grid = 360*3)

R
#>           [,1]       [,2]       [,3]
#> [1,] 0.9291183  0.0338396 -0.3682309
#> [2,] 0.0338396  0.9838446  0.1757970
#> [3,] 0.3682309 -0.1757970  0.9129630
```

The rotational configuration can be fixed. The algorithm finds the rotation in two steps, applying first a Rodrigues rotation as above, then finding the rotation in the 2-dimensional plane perpendicular to the axis just aligned.

```
R <- rotation_to_align(acc_ds,
                       align_to = "-z",
                       align_secondary = "+x",
                       secondary_policy = "max")

R
#>            x          y           z
#> x 0.8998998 -0.2324847 -0.36895950
#> y 0.2588069  0.9656608  0.02276379
#> z 0.3509975 -0.1159744  0.92916667
```

5 References

Xu, T., Xu, X., Xu, D., Zhao, H., 2021. A Novel Calibration Method Using Six Positions for MEMS Triaxial Accelerometer. IEEE Transactions on Instrumentation and Measurement 70, 1-11. https://doi.org/10.1109/TIM.2020.3026024